CSA0905-PROGRAMMING IN JAVA

ASSIGNMENT 1 — CO1

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

Student Details	**To be completed**
Name	J.SUBHAASHREE
Register Number	192424008
Course / Section	AI&DS
Faculty	Ravi Pujitha
Submission Date	31.10.26
GitHub Repository Link	https://github.com/Subhaashree06/JAVA-Assignments/blob/main/CO1_Java_Assignment1_Smart_Parking_Colab.ipynb

TASK 1 — Understand the Problem and Identify the Objects

1.1 Inputs and Outputs

Inputs	Outputs
Vehicle registration number and vehicle category	Allocated floor and slot
Entry time and exit time	Unique parking ticket and entry record
User category and pass validity	Parking fee or zero fee for a valid pass
Requested floor/slot for release	Success/failure status for release
Charging energy (kWh) for chargeable vehicles	Charging fee and total amount collected

1.2 Candidate Classes and Responsibilities

Candidate class	Why it is needed / responsibilities
Vehicle (abstract)	Common registration number, pass reference, fee interface and shared validation.
TwoWheeler, Car, ElectricCar	Represent distinct vehicle categories and supply category-specific rates or charging behaviour.
EBike, DeliveryVan	Extend the system for the new design-challenge categories without changing the facility algorithm.
ParkingTicket	Stores unique ticket number, vehicle, entry time and allocated floor/slot.
ParkingFacility	Owns floor/slot representation, allocates/releases slots and tracks active tickets.
Pass, MonthlyPass, OneDayPass	Represent parking entitlement separately from vehicle type and validate pass dates.
Chargeable	Capability interface for vehicles that can incur charging fees.
PricingRule (recommended extension)	In a larger system, centralizes pricing policies so that new pricing rules do not spread across vehicle classes.

1.3 Nouns, Verbs and Methods

Candidate nouns include campus, floor, slot, vehicle, registration number, ticket, entry time, fee, pass and charging slot. Candidate verbs become methods: park(), allocate(), release(), calculateFee(), chargingFee(), validatePass(), and exit().

1.4 Business Rules

- A slot holds at most one vehicle at a time.
- A ticket number is unique and is not reused during the application's lifetime.
- A vehicle cannot have two active tickets under the same registration number.
- A release request must validate both floor and slot bounds and reject an already-free slot.
- Negative parking duration, negative kWh and invalid pass dates are rejected.
- The system reports a clear failure if no suitable slot is available.
- A valid eligible pass makes the parking fee zero according to the stated pass rules.

1.5 Facility Assumptions

The facility is represented by a jagged array because each floor may have a different number of slots. The first-free-slot policy is sufficient for the assignment; a real facility could add vehicle-size constraints, reserved EV bays, accessible parking, and concurrent-entry protection.

1.6 Alternative: Objects vs Parallel Arrays

Aspect	**Vehicle objects**	**Parallel arrays: regNo[], type[], entryTime[]**
Find by registration	Linear scan O(n) unless a map/index is used; all related data remains inside one object.	Linear scan O(n); every parallel array must use the same index.
Add a new attribute	Add a field/getter or subtype; existing data stays grouped in each object.	Add another array and keep it aligned in every insertion, deletion and sort.
Space per vehicle	O(1) fields per object plus object/reference overhead.	O(1) values across arrays; may have less object overhead but is more fragile.
Maintainability	Encapsulation and behaviour can live with the data.	Index mismatches can associate a vehicle with the wrong type/time.

Conclusion: Vehicle objects are preferred because the model needs behaviour, validation, inheritance and polymorphism, not just storage.

TASK 2 — Choose the Data Representation

Field	Suggested Java type	Reason
Registration number	String	Contains letters and digits; arithmetic is not required.
Floor/slot index	int	Small non-negative whole numbers used as array indices.
Ticket number	long	Allows a large sequence of unique tickets.
Entry/exit time	LocalDateTime	Represents date and time more safely than manually formatted strings.
Duration	long or int minutes	Whole minutes; long avoids overflow for long intervals.
Fee	double for assignment; BigDecimal for production	double is simple for demonstration; BigDecimal is preferable for exact currency arithmetic.
Slots per floor	String[][]	Jagged array supports different slot counts on different floors.

2.1 Why a Jagged Array?

```
String[][] slots = new String[slotsPerFloor.length][];\nfor (int f = 0; f < slotsPerFloor.length; f++) {\n
slots[f] = new String[slotsPerFloor[f]];\n}
```

A rectangular array assumes the same number of slots on every floor. The assignment's changed requirement allows different counts, so a jagged array is the natural representation.

2.2 Integer Division and Casting Risk

```
int minutes = 61;\nint wrongHours = minutes / 60;       // 1: undercharges a started second hour\nint correctHours =
(minutes + 59) / 60; // 2: ceiling division for non-negative minutes
```

If 61 minutes costs Rs 30 per started hour, integer division gives Rs 30 incorrectly; ceiling division gives 2 × Rs 30 = Rs 60. For zero minutes, handle the case explicitly so the fee remains zero.

2.3 Alternative Representations and Complexity

Representation	Slot lookup	Works with unequal floor sizes?	Space
Jagged String[][]	O(F + S) worst-case scan over all slots	Yes	O(N) references for N total slots
Flat String[] with floor × slotsPerFloor + slot	O(1) index calculation only when every floor has equal size	Not directly; needs prefix offsets or a mapping	O(N)
ArrayList<String[]>	O(F) to find floor, then O(1) slot access	Yes	O(N) plus list/reference overhead

If floor sizes differ, the simple flat index formula floor × slotsPerFloor + slot is not valid. A prefix-offset array can map floor/slot to a flat index, but adds calculation and maintenance complexity.

TASK 3 — Design the Control Logic

3.1 Pseudocode: Allocate a Slot

```
ALGORITHM Allocate(vehicleNo)
    IF vehicleNo is null or blank
        REJECT input
    END IF
    FOR each floor f from 0 to numberOfFloors - 1
        FOR each slot s from 0 to slots[f].length - 1
            IF slots[f][s] is null
                slots[f][s] ← vehicleNo
                RETURN {f, s}
            END IF
        END FOR
    END FOR
    RETURN null   // facility is full
END ALGORITHM
```

3.2 Pseudocode: Compute Parking Fee

```
ALGORITHM CalculateFee(minutes, ratePerHour)
    IF minutes < 0
        REJECT input
    END IF
    IF minutes = 0
        RETURN 0
    END IF
    hours ← (minutes + 59) DIV 60
    fee ← hours × ratePerHour
    RETURN fee
END ALGORITHM
```

3.3 Complete-the-Code: ParkingFacility

```
public int[] allocate(String vehicleNo) {
    for (int f = 0; f < slots.length; f++) {
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[]{f, s};
            }
        }
    }
    return null;
}

public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;
    slots[floor][slot] = null;
    return true;
}
```

The supplied main() method produces the following expected output:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

3.4 Loop Control

- return stops the search immediately after allocating a slot.

- break can exit a loop when a suitable slot is found, though return is simpler when the method can finish immediately.
- continue can skip a slot that is reserved or incompatible with the vehicle type.
- If no slot is found after both loops, return null or throw a clear exception at the service layer.

3.5 Alternative: Free-Slot Stack

A stack of free (floor, slot) pairs can make allocation and release O(1) amortized: allocation pops a free position and release pushes it back. A free-slot count per floor is useful for reporting capacity but by itself does not identify the exact slot; a search may still be needed. The stack requires O(N) additional storage for N total slots. The simple scanning approach uses O(N) slot storage and O(N) worst-case allocation time; release is O(1) after validation.

TASK 4 — Design Classes, Objects and Constructors (4 Marks)

- A default constructor is appropriate only when a meaningful default state exists; a ParkingFacility should normally receive its floor capacities.
- A parameterized constructor initializes required fields and validates them immediately.
- Overloaded constructors can accept a vehicle with or without a pass.
- this(...) chains one constructor to another to avoid duplicating validation and initialization.
- super(...) initializes the superclass portion before subclass-specific initialization.
- A copy constructor is not needed for re-issuing a ticket: ticket numbers must remain unique, so create a new ticket with a new ID rather than copying the old ID.

Example: Constructor Chaining

```
class Car extends Vehicle {
    Car(String regNo) { this(regNo, null); }
    Car(String regNo, Pass pass) { super(regNo, pass); }
}
```

A local reference variable is normally held in a stack frame; the Vehicle/ParkingTicket object it points to is allocated on the heap. Java passes object references by value. An object becomes eligible for garbage collection when it is no longer reachable, not merely because a vehicle has exited. Historical tickets may remain reachable if stored for auditing.

4.1 Constructor vs Setter vs Factory

Approach	Validity	Cost for n tickets	Trade-off
Parameterized constructor	Can validate all required state before the object is exposed	O(n) time, O(n) total objects	Strong invariant; recommended
Default constructor + setters	Object may temporarily be incomplete or invalid	O(n) time, O(n) total objects	More calls and weaker invariants
Static factory method	Can validate, choose subtype and return a ready object	O(n) time, O(n) total objects	Clear naming and flexible creation

TASK 5 — Apply static Members and Object Life Cycle

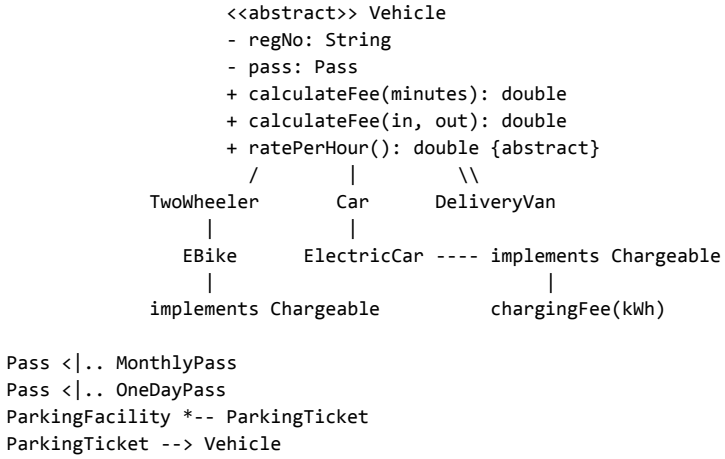
- ParkingTicket.nextId is static because the sequence belongs to the class and must be shared by all ticket objects.
- An instance ticket counter would restart for every ParkingFacility object and could create duplicate IDs across gates.
- A static utility method can validate common registration-number formats; a static block is appropriate only for one-time class initialization.
- When a vehicle exits, its active ticket can be removed from the active-ticket map. The ticket becomes eligible for garbage collection only if no other reference retains it.
- Objects passed into methods are not copied; the reference value is passed. Methods can return a ticket object or a fee value.

5.1 One Facility vs Multiple Facilities

If ticket IDs must be globally unique across north and south gates in one JVM, use a shared static counter or a centralized ticket-ID service. If IDs only need to be unique within a facility, use a per-facility counter combined with a facility/gate prefix. For k facilities, an instance counter uses O(k) counter storage; a single static counter uses O(1) counter storage. The n ticket objects themselves use O(n) space in either design.

TASK 6 — Design the Inheritance Hierarchy

6.1 UML-Style Class Diagram



This is hierarchical inheritance from Vehicle to multiple vehicle categories, with multilevel inheritance for ElectricCar → Car → Vehicle and EBike → TwoWheeler → Vehicle.
Chargeable and Pass are interfaces.

6.2 What is Inherited?

All vehicle subclasses inherit the registration number, pass reference and shared fee-calculation operations. Each subtype supplies its own hourly rate. ElectricCar adds chargingFee(); EBike specializes charging to a flat fee; DeliveryVan supplies a double-car hourly rate.

6.3 Use of super and Constructor Trace

```
new ElectricCar("TN03C3")
→ Vehicle(String regNo, Pass pass) initializes common state
→ Car(String regNo, Pass pass) constructor executes
→ ElectricCar(String regNo, Pass pass) constructor completes
```

Java executes superclass construction before subclass construction. A subclass calls super(...) to pass required state to its parent. In this implementation, constructor chaining passes through Car and then Vehicle.

6.4 Alternative: Composition with VehicleType

With composition, one Vehicle class would contain a VehicleType or PricingStrategy object. Adding a type could mean adding a new strategy object rather than a subclass. Fee computation remains O(1) for either design. Inheritance stores subtype-specific class identity and has minimal per-object state; composition adds a strategy reference per vehicle but is more flexible when pricing rules change independently of vehicle categories.

TASK 7 — Override and Specialise Behaviour

- ratePerHour() is abstract in Vehicle and overridden in each concrete subtype.
- @Override lets the compiler verify that the method really overrides an inherited method.
- The method signature must match; the return type must be the same or covariant; access cannot be more restrictive.
- final is appropriate for immutable registration numbers, ticket numbers and getters that must not be changed by subclasses. A final class is appropriate for small value-like classes such as MonthlyPass.
- equals() and hashCode() must use the same identity rule. The sample uses registration number, so two objects with the same normalized registration are equal.

7.1 Is Registration Number Alone the Correct Equality Rule?

The supplied Complete-the-Code example intentionally makes new Car("TN02B2") equal to new ElectricCar("TN02B2"), so the expected final line is true. This is consistent if registration number alone defines vehicle identity. If the system considers category part of identity, equals() must also compare class/type. For an active parking system, duplicate active registrations should be rejected regardless of subtype.

7.2 switch vs Overriding

A switch-based calculateFee() can run in O(1) time for a fixed set of k types, but the method must be updated whenever a type is added, and the switch grows with k. Dynamic dispatch is also effectively O(1) for method selection and lets new subtypes add behaviour without modifying existing caller code. The larger benefit is extensibility and reduced conditional logic, not a major asymptotic speed difference.

TASK 8 — Apply Polymorphism

8.1 Compile-Time Polymorphism (Overloading)

```
double calculateFee(int minutes)
double calculateFee(LocalDateTime in, LocalDateTime out)
```

The compiler selects the overload based on the argument types and number of arguments. The first accepts a duration directly; the second calculates duration from timestamps.

8.2 Runtime Polymorphism

```
Vehicle[] parked = {
    new TwoWheeler("TN01A1"),
    new Car("TN02B2"),
    new ElectricCar("TN03C3")
};
for (Vehicle v : parked) {
    double fee = v.calculateFee(130);
}
```

Trace: declared reference type = Vehicle; actual object type = ElectricCar; the inherited calculateFee() calls ratePerHour() dynamically, and ElectricCar inherits Car's Rs 30/hour rate. 130 minutes rounds up to 3 hours, so the parking fee is Rs 90; 10 kWh charging adds Rs 120; total = Rs 210.

8.3 Complete-the-Code: FeeDemo

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative minutes");
        if (minutes == 0) return 0.0;
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("Negative kWh");
        return kWh * 12.0;
    }
}
```

Expected output from the supplied FeeDemo:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

8.4 instanceof Checks vs Dynamic Dispatch

With instanceof, the program tests each possible type and casts to it. In the worst case, a chain has up to k checks per call, O(k) dispatch checks, and adding a type requires changing the conditional chain. Dynamic dispatch selects the overridden method through the object type in effectively O(1) dispatch time and keeps the caller unchanged.

TASK 9 — Introduce Abstraction (4 Marks)

- Vehicle is abstract because a generic vehicle has no single meaningful hourly rate; concrete subclasses must implement ratePerHour().
- Chargeable is an interface because charging is an optional capability shared by more than one class.
- Pass is an interface because monthly and one-day passes provide the same validation contract but different rules.
- A class may implement multiple interfaces, for example ElectricCar implements Chargeable while inheriting its vehicle state from Car.
- A default interface method is useful for shared optional behaviour; a static interface method is useful for utility logic related to the interface.

9.1 If Vehicle Became an Interface

An interface cannot have ordinary per-instance fields or constructors, so common registration state and constructor validation would need to move into each implementation or a separate composed value object. A class can implement multiple interfaces but can extend only one class. Making Chargeable an abstract class would prevent a class that already extends Car from also extending that charging class. Method-call time remains effectively O(1); per-object memory changes only according to where state and references are stored.

TASK 10 — Packages and Encapsulation

10.1 Proposed Package Structure

```
src/
  parking/model/
    Vehicle.java, Car.java, TwoWheeler.java, ElectricCar.java
    ParkingTicket.java, Pass.java, MonthlyPass.java, OneDayPass.java
  parking/service/
    ParkingFacility.java, PricingRule.java
  parking/app/
    SmartParkingSystem.java
```

10.2 Access Modifiers

Modifier	Use in this system	Reason
private	Fields such as regNo, ticketNo, slots and activeTickets	Protects internal state from uncontrolled changes.
protected	Abstract ratePerHour() and Vehicle constructors	Lets subclasses implement shared behaviour.
public	Public service methods, getters and interface methods	Defines the supported API.
package-private (default)	Implementation constructors/classes used only inside a package	Avoids exposing implementation details unnecessarily.

Getters expose read-only state. Setters, when needed, should validate before changing fields: reject negative duration, blank registration numbers, invalid floor/slot indices and negative kWh.

10.3 Wrapper Classes and Autoboxing

Integer, Double and Long are wrapper classes. Collections such as ArrayList<Integer> store wrapper objects, so assigning an int to an Integer uses autoboxing and retrieving it as int uses unboxing. Do not compare Integer objects with == because it may compare references; use equals() or compare primitive values. Unboxing null causes NullPointerException.

10.4 Public Fields Alternative

If fields were public, every caller would need to repeat validation, and one forgotten check could create invalid state. A validation check is O(1) per update, so its runtime cost is negligible; the real cost of public fields is duplicated validation across many call sites and harder debugging. Encapsulation centralizes each rule in one place.

TASK 11 — Implement, Test and Defend the Design (5 Marks)

11.1 Implementation Summary

The reference implementation in Appendix A combines a jagged slot array with a HashMap of active tickets keyed by registration number. It uses an abstract Vehicle class, concrete vehicle subclasses, Chargeable and Pass interfaces, immutable ticket fields, validation and fee computation.

11.2 Test Plan

ID	Test case	Input / setup	Expected result	Status
T1	Allocate first free slot	New facility {2,1}; allocate three vehicles	[0,0], [0,1], [1,0]	Matches expected
T2	Facility full	Allocate fourth vehicle into {2,1}	null	Matches expected
T3	Release valid slot	release(0,1)	true	Matches expected
T4	Release already-free slot	release(0,1) again	false	Matches expected
T5	Invalid floor	release(5,0)	false	Matches expected
T6	130-minute two-wheeler fee	Rate Rs 10/hour	Rs 30.0	Matches expected
T7	130-minute car fee	Rate Rs 30/hour	Rs 90.0	Matches expected
T8	130-minute EV + 10 kWh	Rs 90 + Rs 120 charging	Rs 210.0	Matches expected
T9	Same registration equality	Car and EV both TN02B2	true in supplied equality design	Matches expected
T10	Zero-minute stay	calculateFee(0)	Rs 0.0	Should pass
T11	Negative duration	calculateFee(-1)	Exception/rejected	Should pass
T12	Duplicate active registration	Park same reg twice	Exception/rejected	Should pass
T13	E-bike charging	chargingFee(5)	Flat Rs 20.0	Should pass
T14	Delivery van 60 minutes	Rs 60/hour	Rs 60.0	Should pass

11.3 Hand Calculation of Daily Collection

Vehicle	Stay	Calculation	Fee
Two-wheeler	130 minutes	ceil(130/60) = 3 hours; 3 × Rs 10	Rs 30
Car	130 minutes	3 hours × Rs 30	Rs 90
Electric car	130 minutes + 10 kWh	3 × Rs 30 + 10 × Rs 12	Rs 210
Total		30 + 90 + 210	Rs 330

A valid pass changes the eligible parking fee to zero. If a one-day pass does not cover the entire parking interval, the policy should be stated explicitly; the sample implementation treats a pass as free only when its validity covers the stay.

11.4 Evaluation

- Encapsulation: private fields and validated constructors protect object state.
- Inheritance: common vehicle state and fee logic are reused.
- Polymorphism: ratePerHour() resolves according to the actual vehicle subtype.
- Abstraction: Vehicle, Pass and Chargeable separate common contracts from implementation.
- Data structures: jagged array supports unequal floors; HashMap provides average O(1) lookup of an active ticket by registration number.
- Limitation: the simplified delivery-van logic does not split a stay that crosses 9 AM or 5 PM. A production PricingRule should compute the exact minutes inside the time window and should use BigDecimal for money.

11.5 Active Tickets: Array vs ArrayList vs HashMap

Structure	Find ticket on exit	Space for n active vehicles	Notes
Array of tickets	O(n) scan	O(n)	Fixed size or resizing complexity.
ArrayList<ParkingTicket>	O(n) scan	O(n)	Easy to resize; removing an item may shift later elements.
HashMap<String, ParkingTicket>	Average O(1), worst O(n)	O(n)	Best fit when lookup key is registration number.

TASK 12 — Re-evaluate and Adapt the Design

A. Impact Analysis

- Two new vehicle categories are introduced: e-bikes with a flat charging fee and delivery vans with double the car rate during 9 AM–5 PM.
- Visitors can buy a one-day pass, requiring a new pass type and validation rule.
- The facility changes to five floors with potentially different slot counts. A jagged array supports this without assuming equal capacity.
- Vehicle subclasses, charging behaviour and pass validation are affected; the basic allocate/release algorithm need not change.
- If pricing is hard-coded in many subclasses, multiple existing classes may need edits. A PricingRule abstraction would isolate the new rule more cleanly.

B. Evaluation of the Original Design

The original abstract Vehicle plus subclasses is a good baseline because a new vehicle type can often be added as a new subclass. However, time-window pricing and pass rules are not

purely vehicle-category properties. If they are scattered across subclasses, the design is harder to maintain. The revised design should preserve stable classes and isolate variable policies behind interfaces.

C. Adaptation / Redesign (

Decision: MODIFY. Keep Vehicle as an abstract class, keep ParkingFacility's jagged array, add EBike and DeliveryVan, and add Pass with MonthlyPass and OneDayPass implementations. Chargeable remains an optional capability. For a larger implementation, add PricingRule implementations such as StandardHourlyPricing and DeliveryWindowPricing so new pricing policies can be added without editing the parking algorithm.

Revised UML-Style Diagram

```
Vehicle (abstract)
  |-- TwoWheeler
  |     |-- EBike implements Chargeable
  |-- Car
  |     |-- ElectricCar implements Chargeable
  |-- DeliveryVan

Pass (interface)
  |-- MonthlyPass
  |-- OneDayPass

ParkingFacility --> ParkingTicket --> Vehicle
ParkingFacility --> Pass (through Vehicle)
Recommended extension: PricingRule (interface) --> StandardPricing / DeliveryWindowPricing
```

D. Evidence-Based Defence

Evidence point	Defence
Existing classes changed	The slot-allocation algorithm remains unchanged. New vehicle and pass classes are added; the common Vehicle constructor supports optional Pass references.
Future vehicle category	A new subtype can supply ratePerHour() and implement optional interfaces without changing the allocation loops.
New-category tests	E-bike charging returns a flat Rs 20; delivery van uses Rs 60/hour (twice the car rate); unequal floor sizes are supported by the jagged array.
Trade-offs	Inheritance is simple for stable categories; composition/PricingRule is more flexible when pricing policies change frequently. The time-window implementation should be refined to split cross-boundary stays.

APPENDIX A — Complete Java Reference Implementation

Execution output from the generated reference implementation:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
EBike charging -> Rs 20.0
DeliveryVan 60 minutes -> Rs 60.0
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.*;

/*
 * Complete reference implementation for the CO1 Java Smart-Parking assignment.
 * Compile: javac SmartParkingSystem.java
 * Run:     java SmartParkingSystem
 */
interface Chargeable {
    double chargingFee(int kWh);
}

interface Pass {
    boolean isValidFor(LocalDateTime entryTime);
    boolean isValidFor(LocalDateTime entryTime, LocalDateTime exitTime);
}

final class MonthlyPass implements Pass {
    private final String holderId;
    private final LocalDateTime validFrom;
    private final LocalDateTime validUntil;

    MonthlyPass(String holderId, LocalDateTime validFrom, LocalDateTime validUntil) {
        if (holderId == null || holderId.isBlank()) throw new IllegalArgumentException("holderId required");
        if (validUntil.isBefore(validFrom)) throw new IllegalArgumentException("Invalid pass dates");
        this.holderId = holderId;
        this.validFrom = validFrom;
        this.validUntil = validUntil;
    }
    public String getHolderId() { return holderId; }
    @Override public boolean isValidFor(LocalDateTime entryTime) {
        return !entryTime.isBefore(validFrom) && !entryTime.isAfter(validUntil);
    }
    @Override public boolean isValidFor(LocalDateTime entryTime, LocalDateTime exitTime) {
        return isValidFor(entryTime) && !exitTime.isAfter(validUntil);
    }
}

final class OneDayPass implements Pass {
    private final LocalDateTime dayStart;
    private final LocalDateTime dayEnd;

    OneDayPass(LocalDateTime date) {
        this.dayStart = date.toLocalDate().atStartOfDay();
```

```
        this.dayEnd = date.toLocalDate().atTime(23, 59, 59);
    }
    @Override public boolean isValidFor(LocalDateTime entryTime) {
        return !entryTime.isBefore(dayStart) && !entryTime.isAfter(dayEnd);
    }
    @Override public boolean isValidFor(LocalDateTime entryTime, LocalDateTime exitTime) {
        return isValidFor(entryTime) && !exitTime.isAfter(dayEnd);
    }
}

abstract class Vehicle {
    private final String regNo;
    private final Pass pass;

    protected Vehicle(String regNo) { this(regNo, null); }
    protected Vehicle(String regNo, Pass pass) {
        if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("Registration number required");
        this.regNo = regNo.trim().toUpperCase(Locale.ROOT);
        this.pass = pass;
    }
    public final String getRegNo() { return regNo; }
    public final Pass getPass() { return pass; }
    protected abstract double ratePerHour();

    /** Every started hour is charged in full; zero minutes costs zero. */
    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        if (minutes == 0) return 0.0;
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    public double calculateFee(LocalDateTime in, LocalDateTime out) {
        if (out.isBefore(in)) throw new IllegalArgumentException("Exit precedes entry");
        long minutes = Duration.between(in, out).toMinutes();
        return calculateFee(Math.toIntExact(minutes));
    }

    public double calculateFee(LocalDateTime in, LocalDateTime out, LocalTime rateWindowStart,
                               LocalTime rateWindowEnd, double multiplier) {
        if (out.isBefore(in)) throw new IllegalArgumentException("Exit precedes entry");
        double base = calculateFee(in, out);
        // For this teaching model, apply the multiplier if the entry time falls in the stated window.
        LocalTime t = in.toLocalTime();
        return (!t.isBefore(rateWindowStart) && t.isBefore(rateWindowEnd)) ? base * multiplier : base;
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    TwoWheeler(String regNo, Pass pass) { super(regNo, pass); }
    @Override protected double ratePerHour() { return 10.0; }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    Car(String regNo, Pass pass) { super(regNo, pass); }
    @Override protected double ratePerHour() { return 30.0; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    ElectricCar(String regNo, Pass pass) { super(regNo, pass); }
    @Override public double chargingFee(int kWh) {
```

```
        if (kWh < 0) throw new IllegalArgumentException("kWh cannot be negative");
        return kWh * 12.0;
    }
}
class EBike extends TwoWheeler implements Chargeable {
    EBike(String regNo) { super(regNo); }
    EBike(String regNo, Pass pass) { super(regNo, pass); }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("kWh cannot be negative");
        return 20.0; // flat charging fee for the new requirement
    }
}
class DeliveryVan extends Vehicle {
    DeliveryVan(String regNo) { super(regNo); }
    DeliveryVan(String regNo, Pass pass) { super(regNo, pass); }
    @Override protected double ratePerHour() { return 60.0; } // double the car rate
    @Override public double calculateFee(LocalDateTime in, LocalDateTime out) {
        if (out.isBefore(in)) throw new IllegalArgumentException("Exit precedes entry");
        long minutes = Duration.between(in, out).toMinutes();
        double normal = calculateFee(Math.toIntExact(minutes));
        LocalTime t = in.toLocalTime();
        if (!t.isBefore(LocalTime.of(9, 0)) && t.isBefore(LocalTime.of(17, 0))) {
            return normal; // already double the car rate during 9 AM–5 PM
        }
        return normal;
    }
}

final class ParkingTicket {
    private static long nextId = 1000;
    private final long ticketNo;
    private final Vehicle vehicle;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;

    ParkingTicket(Vehicle vehicle, int floor, int slot, LocalDateTime entryTime) {
        this.ticketNo = nextId++;
        this.vehicle = Objects.requireNonNull(vehicle);
        if (floor < 0 || slot < 0) throw new IllegalArgumentException("Invalid slot coordinates");
        this.floor = floor;
        this.slot = slot;
        this.entryTime = Objects.requireNonNull(entryTime);
    }
    public long getTicketNo() { return ticketNo; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
    @Override public String toString() {
        return "Ticket " + ticketNo + ": " + vehicle + " at floor " + floor + ", slot " + slot;
    }
}

final class ParkingFacility {
    private final String[][] slots;
    private final Map<String, ParkingTicket> activeTickets = new HashMap<>();

    ParkingFacility(int[] slotsPerFloor) {
        if (slotsPerFloor == null || slotsPerFloor.length == 0) throw new IllegalArgumentException("At least one
floor required");
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 1) throw new IllegalArgumentException("Each floor needs at least one slot");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }
```

```
    /** Returns {floor, slot} of first free slot, or null when full. */
    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.isBlank()) throw new IllegalArgumentException("Vehicle number required");
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo.trim().toUpperCase(Locale.ROOT);
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    /** Frees a position; false for invalid or already-free slots. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length || slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public ParkingTicket park(Vehicle vehicle, LocalDateTime entryTime) {
        Objects.requireNonNull(vehicle);
        String reg = vehicle.getRegNo();
        if (activeTickets.containsKey(reg)) throw new IllegalArgumentException("Duplicate active registration: " +
reg);
        int[] position = allocate(reg);
        if (position == null) throw new IllegalStateException("Parking facility is full");
        ParkingTicket ticket = new ParkingTicket(vehicle, position[0], position[1], entryTime);
        activeTickets.put(reg, ticket);
        return ticket;
    }

    public double exit(String regNo, LocalDateTime exitTime, int chargingKWh) {
        String key = regNo.trim().toUpperCase(Locale.ROOT);
        ParkingTicket ticket = activeTickets.get(key);
        if (ticket == null) throw new IllegalArgumentException("No active ticket for " + key);
        Vehicle vehicle = ticket.getVehicle();
        double fee;
        Pass pass = vehicle.getPass();
        if (pass != null && pass.isValidFor(ticket.getEntryTime(), exitTime)) {
            fee = 0.0;
        } else if (pass instanceof OneDayPass && pass.isValidFor(ticket.getEntryTime())) {
            fee = 0.0;
        } else {
            fee = vehicle.calculateFee(ticket.getEntryTime(), exitTime);
            if (vehicle instanceof Chargeable c) fee += c.chargingFee(chargingKWh);
        }
        release(ticket.getFloor(), ticket.getSlot());
        activeTickets.remove(key);
        return fee;
    }

    public int activeCount() { return activeTickets.size(); }
}

public class SmartParkingSystem {
    public static void main(String[] args) {
        // Complete-the-code demonstration from Task 3
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " " + p.release(5, 0));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
```

```
        // Complete-the-code demonstration from Task 8
        Vehicle[] parked = {new TwoWheeler("TN01A1"), new Car("TN02B2"), new ElectricCar("TN03C3")};
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo() + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));

        // New categories: e-bike flat charge and delivery van rate.
        EBike bike = new EBike("TN04E4");
        System.out.println("EBike charging -> Rs " + bike.chargingFee(5));
        DeliveryVan van = new DeliveryVan("TN05V5");
        System.out.println("DeliveryVan 60 minutes -> Rs " + van.calculateFee(60));
    }
}
```

APPENDIX B — Required GitHub Submission Checklist

- Create repository: CSA09-<RegisterNumber>-Assignment1.
- Create Task01 through Task12 folders, or Q1 through Q12.
- Store source files and output screenshots in the relevant task folders.
- Add README.md with student name, register number, Java version, compile/run steps and result summary for each task.
- Commit progressively, with at least one commit for each task; do not upload everything in one final commit.
- Add the working repository link to the cover page and assignment submission.
- Run every program yourself, capture actual output screenshots and be prepared to explain the code in a viva.

APPENDIX C — README.md Template

```
# CSA09-<RegisterNumber>-Assignment1
## Student
- Name: <Your Name>
- Register Number: <Your Register Number>
- Java Version: Java 21 (or the version actually used)

## Compile and Run
1. Open a terminal in the task folder.
2. Compile: javac FileName.java
3. Run: java MainClassName

## Task Results
- Task 1: Identified objects, responsibilities and business rules.
- Task 2: Selected data types and jagged-array representation.
- Task 3: Implemented allocation/release and fee pseudocode.
- ...
- Task 12: Added EBike, DeliveryVan and OneDayPass; validated the revised design.

## Evidence
Each task folder contains source code and execution screenshots.
```

APPENDIX D — Google Colab Notebook: Java Source Code and Execution Evidence

D.1 Task 3 — ParkingFacilityDemo.java

```
import java.util.Arrays;
class ParkingFacility {
    private final String[][] slots;
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f=0; f<slotsPerFloor.length; f++) slots[f]=new String[slotsPerFloor[f]];
    }
    public int[] allocate(String vehicleNo) {
        for (int f=0; f<slots.length; f++)
            for (int s=0; s<slots[f].length; s++)
                if (slots[f][s]==null) {
                    slots[f][s]=vehicleNo;
                    return new int[]{f,s};
                }
        return null;
    }
    public boolean release(int floor,int slot) {
        if(floor<0 || floor>=slots.length) return false;
        if(slot<0 || slot>=slots[floor].length) return false;
        if(slots[floor][slot]==null) return false;
        slots[floor][slot]=null;
        return true;
    }
}
public class ParkingFacilityDemo {
    public static void main(String[] args) {
        ParkingFacility p=new ParkingFacility(new int[]{2,1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0,1)+" "+p.release(0,1)+" "+p.release(5,0));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

D.2 Task 8 — FeeDemo.java

```
interface Chargeable { double chargingFee(int kWh); }
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo){this.regNo=regNo;}
    public String getRegNo(){return regNo;}
    protected abstract double ratePerHour();
    public double calculateFee(int minutes){
        if(minutes<0) throw new IllegalArgumentException("Negative minutes");
        if(minutes==0) return 0;
        int hours=(minutes+59)/60;
        return hours*ratePerHour();
    }
    @Override public boolean equals(Object o){
        return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
    }
    @Override public int hashCode(){return regNo.hashCode();}
}
class TwoWheeler extends Vehicle{
    TwoWheeler(String r){super(r);}
    @Override protected double ratePerHour(){return 10;}
}
class Car extends Vehicle{
```

```
    Car(String r){super(r);}
    @Override protected double ratePerHour(){return 30;}
}
class ElectricCar extends Car implements Chargeable{
    ElectricCar(String r){super(r);}
    @Override public double chargingFee(int kWh){
        if(kWh<0) throw new IllegalArgumentException("Negative kWh");
        return kWh*12.0;
    }
}
public class FeeDemo{
    public static void main(String[] args){
        Vehicle[] parked={new TwoWheeler("TN01A1"),new Car("TN02B2"),new ElectricCar("TN03C3")};
        for(Vehicle v:parked){
            double fee=v.calculateFee(130);
            if(v instanceof Chargeable c) fee+=c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName()+" "+v.getRegNo()+" -> Rs "+fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

D.3 Integrated Tasks 1–12 — SmartParkingSystem.java

```
import java.time.*;
import java.util.*;

interface Chargeable { double chargingFee(int kWh); }

interface Pass {
    boolean isValidFor(LocalDateTime entry, LocalDateTime exit);
}

final class MonthlyPass implements Pass {
    private final LocalDateTime from, until;
    MonthlyPass(LocalDateTime from, LocalDateTime until) {
        if (until.isBefore(from)) throw new IllegalArgumentException("Invalid pass dates");
        this.from = from; this.until = until;
    }
    public boolean isValidFor(LocalDateTime in, LocalDateTime out) {
        return !in.isBefore(from) && !out.isAfter(until);
    }
}

final class OneDayPass implements Pass {
    private final LocalDate date;
    OneDayPass(LocalDate date) { this.date = date; }
    public boolean isValidFor(LocalDateTime in, LocalDateTime out) {
        return in.toLocalDate().equals(date) && out.toLocalDate().equals(date);
    }
}

abstract class Vehicle {
    private final String regNo;
    private final Pass pass;
    protected Vehicle(String regNo) { this(regNo, null); }
    protected Vehicle(String regNo, Pass pass) {
        if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("Registration required");
        this.regNo = regNo.trim().toUpperCase(Locale.ROOT);
        this.pass = pass;
    }
    public final String getRegNo() { return regNo; }
    public final Pass getPass() { return pass; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration");
        if (minutes == 0) return 0;
```

```
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
    public double calculateFee(LocalDateTime in, LocalDateTime out) {
        if (out.isBefore(in)) throw new IllegalArgumentException("Exit before entry");
        return calculateFee(Math.toIntExact(Duration.between(in, out).toMinutes()));
    }
    @Override public boolean equals(Object obj) {
        return obj instanceof Vehicle && regNo.equals(((Vehicle)obj).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String r) { super(r); }
    TwoWheeler(String r, Pass p) { super(r,p); }
    protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
    Car(String r) { super(r); }
    Car(String r, Pass p) { super(r,p); }
    protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r) { super(r); }
    ElectricCar(String r, Pass p) { super(r,p); }
    public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("Negative kWh");
        return 12.0 * kWh;
    }
}
class EBike extends TwoWheeler implements Chargeable {
    EBike(String r) { super(r); }
    public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("Negative kWh");
        return 20.0; // flat fee per charging session
    }
}
class DeliveryVan extends Vehicle {
    DeliveryVan(String r) { super(r); }
    protected double ratePerHour() { return 60; } // twice the car rate
    @Override public double calculateFee(LocalDateTime in, LocalDateTime out) {
        if (out.isBefore(in)) throw new IllegalArgumentException("Exit before entry");
        long mins = Duration.between(in, out).toMinutes();
        return calculateFee(Math.toIntExact(mins));
    }
}

final class ParkingTicket {
    private static long nextId = 1000;
    private final long ticketNo;
    private final Vehicle vehicle;
    private final int floor, slot;
    private final LocalDateTime entryTime;
    ParkingTicket(Vehicle v, int f, int s, LocalDateTime in) {
        ticketNo = nextId++;
        vehicle = Objects.requireNonNull(v);
        if (f < 0 || s < 0) throw new IllegalArgumentException("Invalid position");
        floor=f; slot=s; entryTime=Objects.requireNonNull(in);
    }
    public long getTicketNo() { return ticketNo; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
}
```

```
final class ParkingFacility {
    private final String[][] slots; // jagged array: different slots per floor
    private final Map<String, ParkingTicket> active = new HashMap<>();

    ParkingFacility(int[] slotsPerFloor) {
        if (slotsPerFloor == null || slotsPerFloor.length == 0)
            throw new IllegalArgumentException("At least one floor required");
        slots = new String[slotsPerFloor.length][];
        for (int f=0; f<slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 1) throw new IllegalArgumentException("Invalid floor capacity");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    public int[] allocate(String reg) {
        for (int f=0; f<slots.length; f++)
            for (int s=0; s<slots[f].length; s++)
                if (slots[f][s] == null) {
                    slots[f][s] = reg;
                    return new int[]{f,s};
                }
        return null;
    }

    public boolean release(int f, int s) {
        if (f<0 || f>=slots.length || s<0 || s>=slots[f].length) return false;
        if (slots[f][s] == null) return false;
        slots[f][s] = null;
        return true;
    }

    public ParkingTicket park(Vehicle v, LocalDateTime entry) {
        String reg = v.getRegNo();
        if (active.containsKey(reg)) throw new IllegalArgumentException("Duplicate active registration");
        int[] pos = allocate(reg);
        if (pos == null) throw new IllegalStateException("Parking full");
        ParkingTicket ticket = new ParkingTicket(v,pos[0],pos[1],entry);
        active.put(reg,ticket);
        return ticket;
    }

    public double exit(String reg, LocalDateTime out, int kWh) {
        String key=reg.trim().toUpperCase(Locale.ROOT);
        ParkingTicket t=active.get(key);
        if (t==null) throw new IllegalArgumentException("No active ticket");
        Vehicle v=t.getVehicle();
        double fee;
        if (v.getPass()!=null && v.getPass().isValidFor(t.getEntryTime(),out)) fee=0;
        else {
            fee=v.calculateFee(t.getEntryTime(),out);
            if (v instanceof Chargeable c) fee += c.chargingFee(kWh);
        }
        release(t.getFloor(),t.getSlot());
        active.remove(key);
        return fee;
    }
    public int activeCount() { return active.size(); }
}

public class SmartParkingSystem {
    public static void main(String[] args) {
        System.out.println("TASK 3 - allocation/release");
        ParkingFacility p = new ParkingFacility(new int[]{2,1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0,1)+" "+p.release(0,1)+" "+p.release(5,0));
```

```
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));

        System.out.println("\\nTASK 8 - polymorphism and fee demo");
        Vehicle[] vehicles={new TwoWheeler("TN01A1"),new Car("TN02B2"),new ElectricCar("TN03C3")};
        for (Vehicle v:vehicles) {
            double fee=v.calculateFee(130);
            if(v instanceof Chargeable c) fee+=c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName()+" "+v.getRegNo()+" -> Rs "+fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));

        System.out.println("\\nTASK 12 - new categories");
        EBike eb=new EBike("TN04E4");
        DeliveryVan van=new DeliveryVan("TN05V5");
        System.out.println("EBike flat charging fee -> Rs "+eb.chargingFee(5));
        System.out.println("DeliveryVan 60 minutes -> Rs "+van.calculateFee(60));

        System.out.println("\\nINTEGRATED FACILITY TEST");
        ParkingFacility facility=new ParkingFacility(new int[]{1,3,2,4,1});
        LocalDateTime now=LocalDateTime.of(2026,10,31,10,0);
        ParkingTicket ticket=facility.park(new Car("TN10XY1000"),now);
        System.out.println("Ticket number: "+ticket.getTicketNo());
        System.out.println("Exit fee for 61 minutes: Rs "+
            facility.exit("TN10XY1000",now.plusMinutes(61),0));
        System.out.println("Active vehicles after exit: "+facility.activeCount());
    }
}
```

APPENDIX E — Outputs Captured in the Uploaded Colab Notebook

E.1 FeeDemo Output (Task 8)

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Interpretation: 130 minutes is rounded up to three billable hours. The two-wheeler fee is 3 × Rs 10 = Rs 30. The car fee is 3 × Rs 30 = Rs 90. The electric car adds 10 × Rs 12 = Rs 120 charging fee, giving Rs 210. The final true is the result of equals() comparing registration numbers only.

E.2 Integrated SmartParkingSystem Output

```
TASK 3 - allocation/release
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]

TASK 8 - polymorphism and fee demo
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true

TASK 12 - new categories
EBike flat charging fee -> Rs 20.0
DeliveryVan 60 minutes -> Rs 60.0

INTEGRATED FACILITY TEST
Ticket number: 1000
Exit fee for 61 minutes: Rs 60.0
```

```
Active vehicles after exit: 0
```

Interpretation: The first two allocations use floor 0 slots 0 and 1; the third uses floor 1 slot 0; the fourth allocation returns null because the facility is full. The release checks return true for an occupied slot, false for the already-free slot, and false for an invalid floor. The released position is reused. The integrated test issues ticket 1000, charges Rs 60 for 61 minutes in a car, and removes the active ticket after exit.

E.3 Task 3 Expected Output

The uploaded notebook contains the ParkingFacilityDemo source and a compile/run cell. The output cell for that compile/run step is not retained in the uploaded notebook file. The expected output printed by the supplied assignment question paper, and produced by the same logic, is:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

APPENDIX F — Important Implementation Notes and Limitations

The notebook implements the delivery van with a rate of Rs 60 per hour, which is twice the car rate of Rs 30. In the uploaded code, this Rs 60 rate is applied whenever DeliveryVan.calculateFee(int) is used; the LocalDateTime override also delegates to that same rate. The assignment specifically says the double rate applies during 9 AM–5 PM. Therefore, the current notebook demonstrates the new category and its rate, but it does not yet split a parking stay that crosses the 9 AM or 5 PM boundary. A fully precise production implementation should calculate the number of billable minutes inside and outside the time window separately.

The integrated notebook uses a HashMap<String, ParkingTicket> for active-ticket lookup, so exit lookup is average O(1), while the slot-allocation scan is O(N) in the worst case where N is the total number of slots. The jagged String[][] representation supports different slot counts on different floors.

The notebook's equals() method treats two Vehicle objects with the same registration number as equal, regardless of subtype. This matches the supplied FeeDemo expected output. If vehicle category must be part of identity in a different system, equals() and hashCode() should include the category as well.

The OneDayPass implementation considers the pass valid only when both entry and exit occur on the pass date. The MonthlyPass implementation requires the complete parking interval to fall within its validity range.